Practical : 7

Newton’s Forward Interpolation

Aim: Program to implement Newton’s Forward Interpolation formula.

Code:

```
#include <stdio.h>

#define MAXN 20

float absolute_value(float number) {
    return number < 0.0f ? -number : number;
}

float larger_value(float first, float second) {
    return first > second ? first : second;
}

int main(void) {
    int n;
    float x[MAXN], difference[MAXN][MAXN];
    float value, h, u, term, result;

    printf("Newton Forward Interpolation\n");
    printf("Enter number of data points (2-%d): ", MAXN);
    if (scanf("%d", &n) != 1 || n < 2 || n > MAXN) {
        printf("Invalid number of data points.\n");
        return 1;
    }

    printf("Enter x and y values:\n");
    for (int i = 0; i < n; i++) {
        printf("x[%d] y[%d]: ", i, i);
        if (scanf("%f %f", &x[i], &difference[i][0]) != 2) {
            printf("Invalid input.\n");
            return 1;
        }
    }

    h = x[1] - x[0];
    if (absolute_value(h) < 1e-12f) {
        printf("Invalid data: x values must be distinct.\n");
        return 1;
    }
    for (int i = 2; i < n; i++) {
        if (absolute_value((x[i] - x[i - 1]) - h) >
            1e-9f * larger_value(1.0f, absolute_value(h))) {
            printf("Invalid data: x values must be equally spaced.\n");
            return 1;
        }
```

```
}

for (int order = 1; order < n; order++) {
    for (int row = 0; row < n - order; row++) {
        difference[row][order] =
        difference[row + 1][order - 1] - difference[row][order - 1];
    }
}

printf("\n=======================================\n");
printf("       Forward Difference Table\n");
printf("=======================================\n");
printf(" %10s ", "x");
for (int order = 0; order < n; order++) {
    char label[20];
    if (order == 0)
        snprintf(label, sizeof(label), "y");
    else
        snprintf(label, sizeof(label), "Δ^%d y", order);
    printf("| %10s ", label);
}
printf("|\n");
printf("---------------------------------------\n");

for (int row = 0; row < n; row++) {
    printf(" %10.4f ", x[row]);
    for (int order = 0; order < n - row; order++) {
        printf("| %10.4f ", difference[row][order]);
    }
    for (int order = n - row; order < n; order++) {
        printf("| %10s ", "");
    }
    printf("|\n");
    printf("---------------------------------------\n");
}

printf("\nNewton Forward Polynomial:\n");
printf("P(x) = %.6f", difference[0][0]);
for (int order = 1; order < n; order++) {
    if (absolute_value(difference[0][order]) < 1e-12f) {
        continue;
    }
    printf("\n       + (%.6f)", difference[0][order]);
    for (int factor = 0; factor < order; factor++) {
        printf("(x - %.6f)", x[factor]);
    }
    printf("/(%d! * %.6f^%d)", order, h, order);
}
printf("\n");

printf("\nEnter the x value to interpolate: ");
if (scanf("%f", &value) != 1) {
```

```
        printf("Invalid input.\n");
        return 1;
    }

    u = (value - x[0]) / h;
    result = difference[0][0];
    term = 1.0f;

    for (int order = 1; order < n; order++) {
        term *= (u - (order - 1)) / order;
        result += term * difference[0][order];
    }

    printf("Interpolated value at x = %.4f is %.6f\n", value, result);
    return 0;
}
```

Output:

```
Newton Forward Interpolation
Enter number of data points (2-20):
5
Enter x and y values:
x[0] y[0]: 4 24
x[1] y[1]: 5 60
x[2] y[2]: 6 120
x[3] y[3]: 7 180
x[4] y[4]: 8 240

=====================================
      Forward Difference Table
=====================================
         x |         y |     Δ^1 y |    Δ^2 y |    Δ^3 y |    Δ^4 y |
-------------------------------------
    4.0000 |   24.0000 |   36.0000 |   24.0000 |  -24.0000 |   24.0000 |
-------------------------------------
    5.0000 |   60.0000 |   60.0000 |    0.0000 |    0.0000 |           |
-------------------------------------
    6.0000 |  120.0000 |   60.0000 |    0.0000 |           |           |
-------------------------------------
    7.0000 |  180.0000 |   60.0000 |           |           |           |
-------------------------------------
    8.0000 |  240.0000 |           |           |           |           |
-------------------------------------

Newton Forward Polynomial:
P(x) = 24.000000
      + (36.000000)(x - 4.000000)/(1! * 1.000000^1)
      + (24.000000)(x - 4.000000)(x - 5.000000)/(2! * 1.000000^2)
      + (-24.000000)(x - 4.000000)(x - 5.000000)(x - 6.000000)/(3! * 1.000000^3)
      + (24.000000)(x - 4.000000)(x - 5.000000)(x - 6.000000)(x - 7.000000)/(4! * 1.000000^4)

Enter the x value to interpolate: 12
Interpolated value at x = 12.0000 is 1320.000000
```